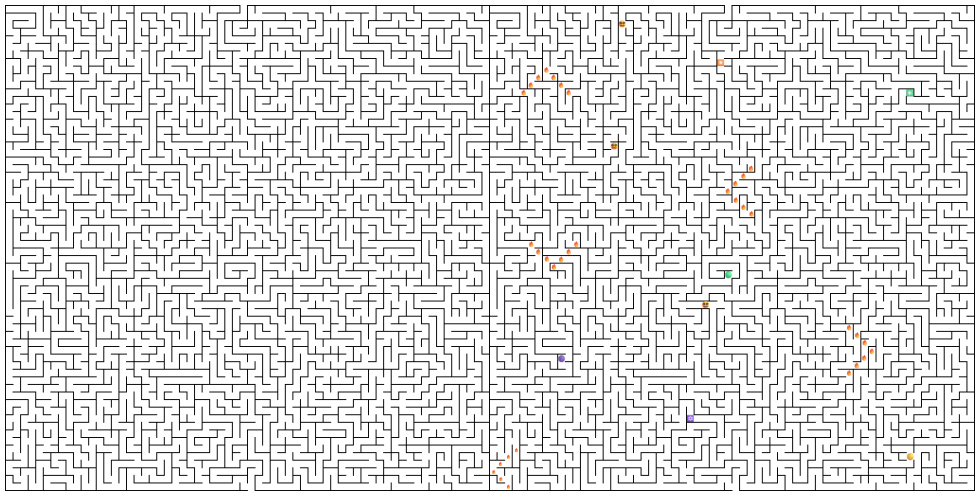
Group 5:
Jason Quach, Mario Chavez, Lizzie Saucedo, Aliza Salman, Phillip Pineda

Group 5 Project Check In #2

Hazards in a Maze

Overview/Goal:

The goal of the Python program is to load a Maze into Python, visualize the solution, then load hazards in the Maze exhibiting the program's function to identify and demonstrate that they function as intended.



***MAZE_0.png and MAZE_1.png*

Thought Process:

What Algorithm?:

The pathingfinding implementation is built off the first Check In's program, meaning we will be using **Breadth First Search (BFS)**.

We will use BFS rather than A* Search as it works better with the Mazes provided. This is because the images are analyzed in a grid-like manner, assigning pixels as either FREE(0) or WALL(1) cells. With no intermediate cost values, the environment is unweighted, so every cell is of equal value. BFS becomes more ideal in this scenario since every step will cost the same as BFS guarantees to find the shortest path possible by moving in all directions equally from the start; however, it can be "blind" and spend more time exploring dead ends. As it expands and explores everything, it will explore the entire reachable state space even if the "exit/goal" is reached. A* does have potential to be better in more complex mazes.

- **"A* Search" acts like BFS, but calculates distance already travelled plus an estimate on how much further it needs to travel for the goal. (Heuristic function, better when cells that are being counted have different costs.) Also, as we move on to identifying Hazards, it will NOT work well in that scenario. A* does not help demonstrate the hazard's functions.*

Group 5:
Jason Quach, Mario Chavez, Lizzie Saucedo, Aliza Salman, Phillip Pineda

Image Processing and Mapping a Grid Structure

To have the Maze readable for BFS to navigate, the following steps were taken:

- **Grayscale Thresholding**:
  - The program utilizes a threshold of 128 to differentiate between the paths and barriers of the Maze. Through checking RGB values of the pixels, any value above the threshold is a traversable path, labeled as FREE. Any value below is labeled as WALL. (mentioned in #1.)
- **Automatic Start and Goal Detection**:
  - Rather than hard-coding the starting pixel coordinates (x,y) of the Maze, the program scans the top and bottom rows of the PNG, identifying the “entrance” and “exit” by locating specific coordinates where the pixel exceeds the threshold 128. This allows the Maze to be more adaptable for future Mazes loaded into the Python script.
- **Data Structures?:**
  - To store the image, we resort to a 2D NumPy array (grid), allowing for constant-time O(1) to assist with the BFS algorithm if a neighboring cell is a wall or a path.

Group 5:
Jason Quach, Mario Chavez, Lizzie Saucedo, Aliza Salman, Phillip Pineda

Implementation of Hazards?:

With Check In 2, we are now provided an overlay Maze for Maze_0, Maze_1. This image has multiple hazards implemented with different meanings for each. Our goal here is to load the hazards and show that they behave as intended. Thinking about this, we can:

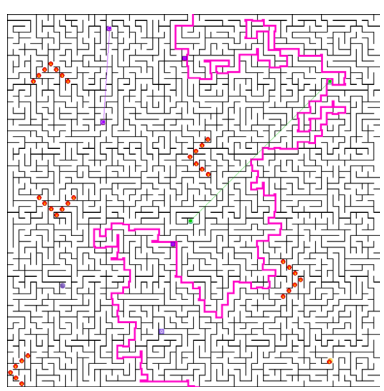
- Keep our current code with the BFS Algorithm
- Build a hazard system on top of our current program with a simulated “agent walk” that goes through the BFS Path, then triggers on specific hazard icons/emojis.

We will have 2 new imports, ImageDraw from our pre-existing PIL (Pillow) and ndimage from scipy. ImageDraw will allow us to overlay the hazards while ndimage will assist with touching the colored pixels of the specific hazards. The Hazards provided consist of:

- 🔥 = Death Pit
    - User/Agent dies and respawns at the start of the Maze.
- Teleport Pads =
    - 🟢 -> ✳️
    - 🟡 -> 🔆
    - 🟣 -> 🔯
        - Warps the User/Agent to the Teleport pads after stepping on the color.
- 😵‍💫 = Confusion Traps
    - Confuses the User/Agent, turns them in a different direction.

These are all determined automatically, except for the Confusion trap. The program itself determines the RGB colors of each hazard by scanning every pixel in the Maze, checking if certain pixel RGB values fall within a specific range. When it does, it gets flagged. This is where ndimage comes in. It puts all the pixels with the RGB values into one cluster, becoming the Hazard emoji loaded in the maze with the center of the cluster being the location of the hazard. This unfortunately does not work well with Confusion traps, due to the details of the face, so it’s harder to generalize its color compared to the Death Pit and Teleport Pads. To remedy this, it is the only hazard that we specifically input the pixel location of.

***Note: Originally, all the hazards were going to have hard-coded locations in the program, but decided against it since autonomous detection seemed better, especially if we’re given another Maze with differently positioned hazards. So we used Claude to help us determine a way to do it. This lets us only need to find the exact coordinates of Confusion Traps in future Mazes.*



A Hazard log is also generated at the end

Conclusion:

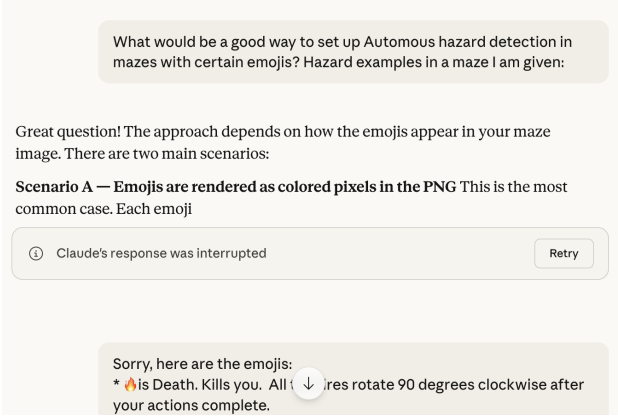
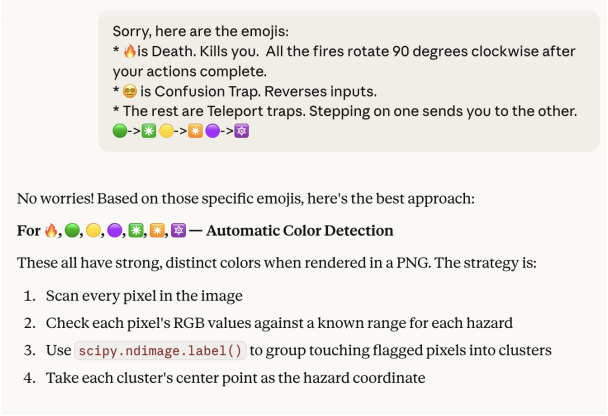
Through this Check In We successfully:

- Automate Hazard Detection: Using ndimage, we are able to cluster up similar color pixels through their rgb values and determine what hazard they are. (Bar Confusion Traps due to their more complex image)
- Functional Agent: We implemented an “agent” who walks through the generated BFS path and encounters and interacts with the hazards on the path.
- Hazard Report: We implemented a hazard report to better display any interactions an “agent” may have.

In conclusion, we were able to make the necessary additions to the initial Check_In_1 code to satisfy the hazard detection system required in Check_In_2. We integrated the system into our Bread First Search (BFS) frame work, utilizing ndimage from scipy for pixel rgb recognition and ImageDraw from PIL (Pillow), moving past static-maze solving and having a more interactive environment where an “agent” or user is moving through the BFS path. The semi-automous detection system leaves for better adaptability for other mazes. The improvements made will lead to us having to improve the pathing logic more for future checkins.

Group 5:
Jason Quach, Mario Chavez, Lizzie Saucedo, Aliza Salman, Phillip Pineda

Resource(s):

- https://pypi.org/project/pillow/
- https://pillow.readthedocs.io/en/stable/reference/ImageDraw.html
- https://pypi.org/project/scipy/
- https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.label.html
- https://docs.scipy.org/doc/scipy/reference/ndimage.html
- https://numpy.org/doc/stable/user/basics.indexing.html
- https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/
- https://www.geeksforgeeks.org/dsa/a-search-algorithm/
- https://www.geeksforgeeks.org/artificial-intelligence/a-algorithm-and-its-heuristic-search-strategy-in-artificial-intelligence/
- https://www.geeksforgeeks.org/dsa/difference-between-best-first-search-and-a-search/

**Utilized Claude for helping setup the autonomous pixel detection so we wouldn't have to hard-code the hazard locations.